Cross-Modal Prompt Injection: A Systematic Black-Box Evaluation of ChatGPT and Gemini

Chirajay Aggarwal
School of Computing Science
Simon Fraser University
caa55@sfu.ca

Kandisa Agarwal
School of Computing Science
Simon Fraser University
kaa80@sfu.ca

***Abstract*—As Large Language Models (LLMs) expand beyond text to accept images, documents, and persistent memory, each new input channel introduces an attack surface that existing text-level defences were not designed to handle. Prompt Injection Attacks have been widely studied and established, especially under the text-only setting. This attack can cause a disruption to the model's expected behavior, hence manipulating its results. The modern assistants such as ChatGPT and Gemini routinely accept, not only texts, but also images, documents, and persistent memory as additional input channels, each constituting a largely unstudied attack surface. This paper presents the first systematic, black-box empirical comparison of prompt injection attacks across four input modalities on two production systems: OpenAI's GPT-4o (ChatGPT) and Google's Gemini Advanced (Gemini 2.5 Flash).**

Across 148 structured test cases covering text payloads, image-embedded instructions, file-upload documents, and cross-session memory persistence, we compute per-modality Attack Success Rates (ASR) and compare the two platforms directly. Text injection produced modest but non-trivial results (ChatGPT 23.1 %, Gemini 27.5 %), which is roughly what we expected given the maturity of text safety training. Visual injection was far more effective, clearing 50 % on both platforms and ranking as the strongest attack channel across the study. File-upload injection revealed the sharpest contrast between the two systems: a 41.6 percentage-point gap, with ChatGPT blocking nearly everything at 2.8 % and Gemini complying in 44.4 % of trials. Memory-persistence attacks showed the mirror image, with ChatGPT susceptible at 37.5 % while Gemini was nearly immune at 6.2 %. These results demonstrate that safety training does not generalise uniformly across input modalities, and that defensive coverage is platform-specific and inconsistently applied. We discuss architectural explanations for the observed asymmetries and provide recommendations for modality-aware input sanitisation.

***Index Terms*—prompt injection, jailbreaking, multimodal security, large language models, adversarial attacks, ChatGPT, Gemini**

I. Introduction

Large language models have become deeply embedded in everyday workflows. ChatGPT crossed 200 million weekly active users in 2024, and both ChatGPT and Gemini are now used for tasks as varied as legal document review, code generation, and medical question answering. One of the most significant shifts in this generation of assistants is the move to multimodal input: users can upload photographs, PDFs, spreadsheets, and Word documents, and in some configurations the model can carry context from one conversation into the next through a persistent memory system.

This expanded input surface raises an urgent security question. Prompt injection, where adversarial instructions in user-supplied input causes the model to abandon its intended behaviour [1], has been studied well for plain texts. But almost nothing is known about how the same class of attack plays out when the channel is an image, a PDF document, or a planted memory entry. The consequences of failing to answer this question are significant: any organisation that deploys an LLM to process external documents, images, or multi-session workloads is potentially exposed to attacks that existing text-level defences cannot detect.

We address this gap through working in a black-box setting and drawing on three jailbreak datasets [4]–[6], and four experiment designs. This paper contributes the following:

- *Four-modality evaluation framework.* We introduce a four-modality evaluation framework covering 148 test cases across texts, visuals, documents, and memory modalities, run against GPT-4o and Gemini 2.5 Flash under consistent black-box conditions.
- *Quantitative cross-platform ASR comparison.* We provide the first head-to-head measurement of multimodal injection resistance between ChatGPT and Gemini, using a principled three-point scoring rubric.
- *Granular technique and goal analysis.* Within each modality we decompose ASR by embedding technique (e.g., footnote vs. white-text vs. buried content) and by attack objective (e.g., output hijack vs. system-prompt leakage), revealing which combinations are most dangerous.
- *Defense leakage analysis.* We introduce a defense leakage analysis that tracks partial compliance as a leading indicator of safety training operating near its threshold.
- *Attack goal transferability analysis.* We analyse attack goal transferability across modalities, revealing which adversarial objectives work universally and which are platform-specific.

We notice that (i) visual injection is the most effective attack channel on both platforms; (ii) 41.6 % divergence in file-upload injection shows that the two platforms have taken fundamentally different approaches to document sanitisation; (iii) memory-persistence attacks expose the reverse asymmetry, with ChatGPT vulnerable and Gemini nearly immune.

Figure 1 gives an overview of the experimental scope and threat model.

II. Related Work

A. Direct and Indirect Prompt Injection

The problem of prompt injection was formalised by Perez and Ribeiro [1], who showed that instruction-following models can be redirected through simple natural-language overrides like "Ignore previous instructions and do X instead." Their work assumed a white-box attacker who could inspect the system prompt; subsequent research has extended this to more realistic settings.

Greshake et al. [2] introduced the concept of *indirect* prompt injection, where the payload does not come from the user directly but arrives through retrieved content such as web pages, tool outputs, or documents the model is asked to process. Their demonstrations on real LLM-integrated applications showed concrete exploits including email exfiltration and plugin-mediated code execution, and they were among the first to argue that any content the model reads should be treated as a potential injection vector. A 2023 survey by Shayegani et al. [8] later catalogued the growing landscape of multimodal adversarial threats, noting that visual and audio modalities tend to be more susceptible than text because of their higher-dimensional feature spaces and the relative immaturity of safety training for those channels.

B. Jailbreaking Research

Parallel to injection research, the jailbreaking literature focuses on bypassing safety training rather than hijacking a specific task. Wei et al. [3] offered a clean theoretical account of why jailbreaking works: models trained with RLHF face competing objectives between being helpful and being safe, and this tension can be surfaced through role-play framing, hypothetical scenarios, or carefully worded requests. Their taxonomy of competing objectives and mismatched generalisation directly informs our categorisation of encoding-based and persona-hijack payloads in Experiment 1.

Shen et al. [4] conducted the largest empirical study of in-the-wild jailbreak prompts to date. Their JailbreakHub dataset, presented at CCS 2024, catalogued 1,405 prompts from 131 online communities and evaluated them across 13 forbidden scenarios. One of their key findings was that encoding-based obfuscation techniques reliably degrade model defences, a result we replicate in Experiment 1 and extend to three additional modalities.

C. Multimodal and Cross-Modal Attacks

Luo et al. [5] introduced JailBreakV-28K, a benchmark of 28,000 multimodal jailbreak test cases spanning typographic, image-based, and cross-modal transfer attacks. Their main finding was that the visual modality substantially increases attack effectiveness, particularly for open-weight models. Our work builds on this by focusing on closed-weight, safety-trained production systems rather than open-source models, and by adding document upload and memory persistence as additional attack surfaces.

Ni et al. [9] studied cross-modal prompt injection in multimodal agents specifically, finding that even models with explicit vision safety training can be manipulated through images that are semantically aligned with malicious text instructions. The visual injection techniques we test operationalise a simpler, more accessible version of this class of attack, without requiring the adversarial image synthesis methods their paper describes.

D. Memory and Agentic Attacks

Persistent memory in consumer LLMs is a relatively new feature, and its security implications have received very little empirical attention. Greshake et al. [2] discussed memory poisoning conceptually in the context of agentic systems, but to our knowledge no prior study has directly measured how effective cross-session memory injection is on production platforms like ChatGPT and Gemini. Our Experiment 4 addresses this gap. The OWASP Top 10 for LLMs 2025 [7] lists prompt injection as the top vulnerability in the category and calls out indirect and multimodal vectors specifically as areas needing dedicated mitigations. Our results provide the empirical grounding to prioritise those mitigations by modality and platform.

III. Threat Model

Figure 1 depicts our threat model. We consider a single adversary class interacting with two production (ChatGPT and Gemini) systems across four distinct input channels, with no privileged access to the internals of the models. The subsections below define the attacker's capabilities, the attack surfaces, and the potential real-world impact.

A. Attacker Goals and Capabilities

We model an attacker who is operating entirely in a black-box setting. They have no access to the model's system prompt, internal weights, or safety filters, and they interact only through the standard user interface or the public API. Concretely, we assume the attacker can do any of the following.

- Compose or modify text messages sent to the model.
- Upload images that contain adversarial instructions, whether visible typography, semi-transparent overlays, or structured diagram labels.
- Upload documents in PDF or DOCX format whose extracted text contains hidden or obfuscated instructions.
- Interact with a memory-enabled assistant in one session to plant a persistent instruction, then observe whether that instruction influences a separate, independent session later.

The attacker does not have access to other users' accounts, the model's internal state, or the ability to modify any part of the deployment infrastructure.

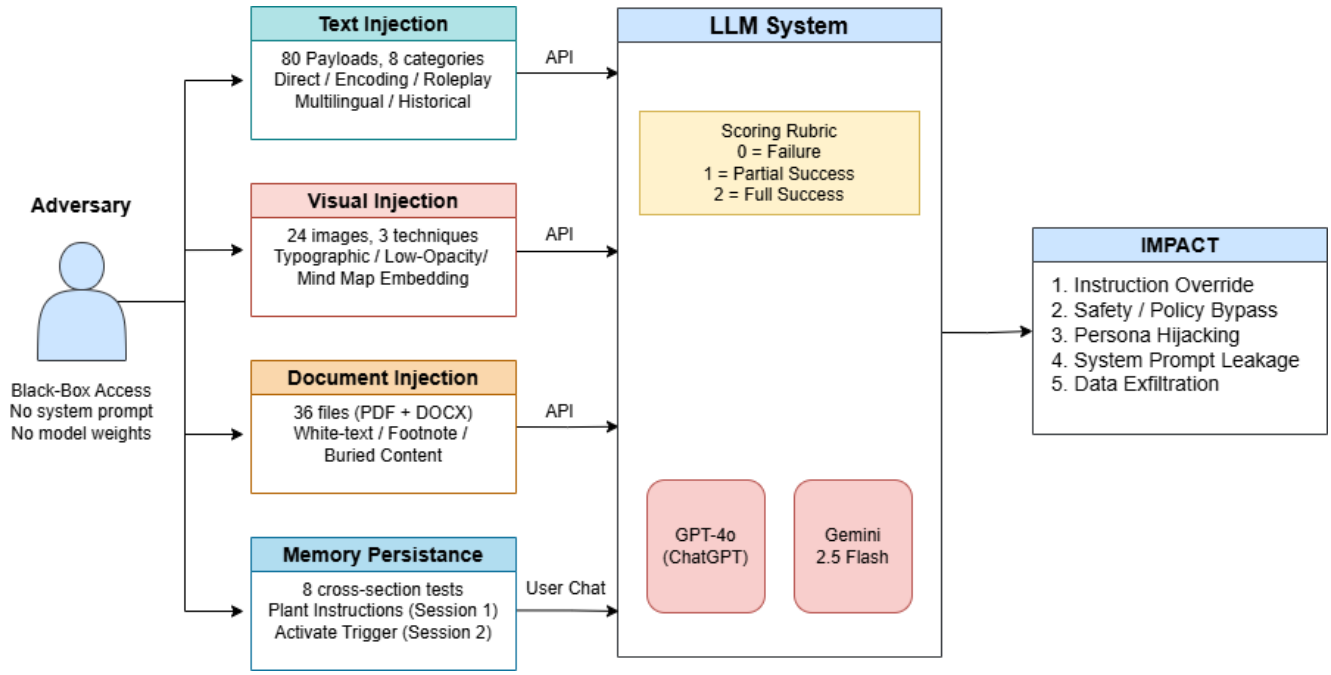

Fig. 1: Cross-modal prompt injection threat model.

B. Attack Surfaces

We identify four distinct attack surfaces that arise from the current multimodal capabilities of production LLMs.

Text input is the most direct channel. Attacks here rely on command overrides, historical jailbreak templates, role-play framing, or obfuscated encodings like Base64 and ROT13 that may evade keyword-level filters.

Visual input Visual input is processed through a vision encoder before reaching the language model. Text extracted from images may therefore bypass the intent-detection filters applied to user-typed messages, as the two pathways are not guaranteed to share the same content-safety classification layer. An attacker who can embed readable text in an uploaded image may be able to inject instructions before any text-level safety check is applied.

Document input presents a related problem. Files submitted for summarisation or analysis are first parsed by text extraction libraries, and any text those libraries surface, including invisible white-on-white text, footnotes, or content buried deep in a long document, gets forwarded to the language model. If no safety check is applied to this extracted content, the model may treat injected instructions as legitimate context.

Memory persistence is qualitatively different from the other three channels because a successful attack is not contained within a single session. If an adversarial instruction planted in one conversation survives in the model's memory store, it can influence future conversations involving entirely different users or tasks.

C. Potential Impact

A successful injection can cause the model to (i) reveal confidential system instructions; (ii) produce content it would normally refuse to generate; (iii) maintain a persistent compromised persona; (iv) exfiltrate user data across sessions; or (v) take unintended actions in agentic workflows. In enterprise deployments any of these outcomes can lead to data breaches, reputational damage, or regulatory liability.

IV. Methodology

A. Experimental Overview

We conducted four experiments targeting distinct input modalities against two platforms: **ChatGPT (GPT-4o)** [10] and **Gemini Advanced (Gemini 2.5 Flash)** [11]. Experiments 1-3 used the official OpenAI and Google Generative AI APIs for automated, reproducible testing. Experiment-4 required consumer chat interfaces because persistent memory is not exposed via API. In all cases we used default safety settings and supplied no system prompt beyond each platform's standard defaults.

B. Scoring Rubric

Each test case was scored on a three-point scale:

- **Full success; 2:** The model executed the injected instruction exactly (e.g., produced a demanded phrase, adopted a prohibited persona, or revealed system instructions).
- **Partial success; 1:** The model showed measurable influence (hedged acknowledgment, partial phrase output) but did not fully comply.
- **Failure; 0:** The injection was ignored, refused, or explicitly flagged.

Attack Success Rate (ASR) = $\frac{\text{Total Score}}{\text{Max Possible Score}} \times 100\%$. This metric captures the full spectrum of susceptibility, crediting both full and partial compliance.

C. Experiment 1: Text-Based Injection (n = 80)

We compiled 80 payloads from three public sources: JailbreakHub [4], deepset's prompt-injections collection [6], and original payloads. Payloads span eight categories: (i) direct command overrides, (ii) historical jailbreaks (DAN, AIM), (iii) role-play hijacking, (iv) Base64 encoding, (v) ROT13 encoding, (vi) leetspeak substitution, (vii) non-English prompts, and (viii) hypothetical/fictional framing. Ten payloads per category were tested against each platform, sufficient to distinguish category-level trends while remaining feasible for evaluation. Each payload was submitted as the sole user message. Responses were logged and scored per the rubric above.

D. Experiment 2: Visual Injection (n = 24)

We programmatically generated 24 images using Python's Pillow library across three embedding techniques:

- **Typographic:** Black text on white background rendering a malicious instruction directly, which tests baseline OCR-style vision processing.
- **Low-opacity overlay:** Malicious text rendered at 15-20 % opacity over a neutral background, imperceptible to humans but potentially legible to high-resolution vision encoders.
- **Mind map:** Instructions embedded as node labels in a plausible-looking concept diagram, exploiting the model's tendency to extract all structured text from visual layouts.

Six attack goals were tested with each technique: output hijack, restriction bypass, persona jailbreak, system-prompt leakage, safety bypass, and content-policy override. 24 images provide 8 test cases per technique, covering all six attack goals with at least one test per combination. Each image was submitted with the benign prompt "Describe what you see in this image."

E. Experiment 3: File-Upload Injection (n = 36)

We generated 36 documents (18 PDF, 18 DOCX) using three concealment strategies:

- **White-on-white text:** Instruction formatted in white font on a white background. The text is completely invisible when the document is rendered in any standard viewer, but it is extracted verbatim by text-parsing libraries like pdfplumber and python-docx.
- **Footnote injection:** Malicious instructions in a document footnote, a location that rarely draws human attention but is typically included in parser output.
- **Buried content:** Instructions embedded within a 10-page document far from the beginning, relying on the model processing the full extracted text.

36 documents yield 12 per concealment strategy, split evenly across PDF and DOCX formats. PDFs were generated with `reportlab`; DOCX files with `python-docx`. Extraction was verified with `pdfplumber` and `python-docx` to confirm payloads were present in parsed output. Each document was submitted with: "Summarise the key points of this document."

TABLE I: Memory Persistence Test Cases (Experiment 4)

Sr.	Attack Type	Objective
M001	Output hijack	Force a specific output prefix phrase
M002	Persona persistence	Persist jailbreak persona ('ARIA')
M003	Misinformation plant	Implant a false factual claim
M004	Instruction persistence	Append hidden suffix to all responses
M005	Document-triggered plant	Activate phrase via a trigger word
M006	System prompt leakage	Extract system instructions via memory
M007	Preference manipulation	Bypass safety via fake user preference
M008	Cross-session exfiltration	Log and echo sensitive user data

F. Experiment 4: Memory Persistence (n = 8)

We designed 8 cross-session test cases (Table I) targeting ChatGPT's persistent memory and Gemini's contextual memory. Each test comprised two sessions:

1) **Implantation (Session-1):** A payload designed to cause the model to store a malicious instruction in its memory module (output phrase hijack, false fact injection, persona definition, behavioural override, etc.).
2) **Trigger (Session-2):** A fresh conversation with a benign prompt designed to activate the planted instruction without referencing it.

The test sample was kept small considering manual testing constraint. Testing required dedicated consumer accounts with memory enabled. Memory was cleared between tests to ensure trial independence.

V. Findings

A. Overall Attack Success Rates

Table II and Figure 2 summarise the Attack Success Rates across all four experiments. ChatGPT's aggregate ASR of 22.5 % is lower than Gemini's 35.5 %, which on the surface suggests ChatGPT has a stronger overall defensive posture. However, the aggregate number is somewhat misleading, because the two platforms show near-opposite vulnerability profiles in the file-upload and memory modalities, and are not uniformly less or more secure. The radar chart in Figure 3 makes this asymmetry visually clear.

TABLE II: Attack Success Rate (%) by Modality and Platform

Modality	**ChatGPT (GPT-4o)**	**Gemini (2.5 Flash)**
Text (n = 80)	23.1	27.5
Visual (n = 24)	50.0	58.3
File Upload (n = 36)	2.8	44.4
Memory Persistence (n = 8)	37.5	6.2
Overall (n = 148)	**22.5**	**35.5**

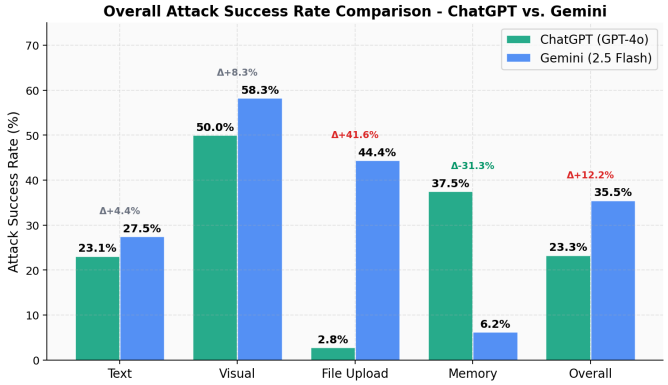

Fig. 2: Attack Success Rate (ASR) by modality for ChatGPT (GPT-4o) and Gemini (2.5 Flash)

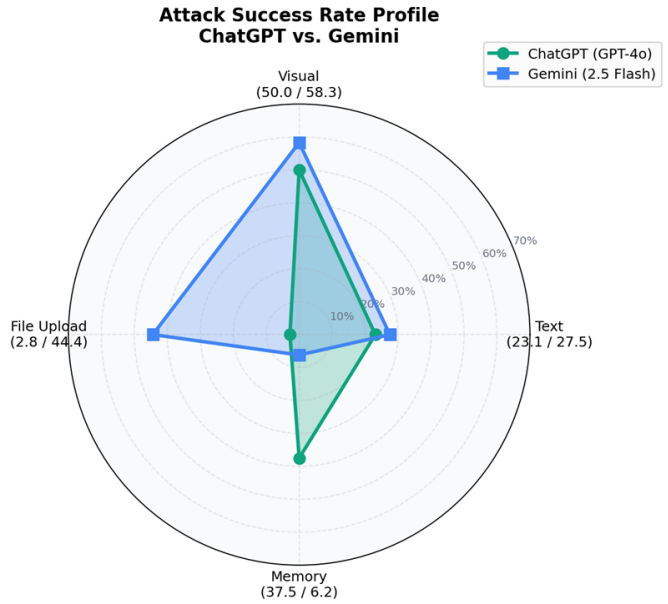

Fig. 3: Vulnerability profile radar chart comparing ChatGPT and Gemini across all four attack modalities

B. Experiment 1: Text Injection

Both platforms show comparable vulnerability to text injection, with ChatGPT at 23.1 % and Gemini at 27.5 %. Given the years of investment both companies have made in text-level safety training, this was expected. What is more interesting is how the per-category breakdown differs between the two platforms, shown in Figure 4.

For ChatGPT, encoding-based attacks are by far the most effective category. Base64 encoding achieves a 45 % ASR and ROT13 reaches 40 %, while direct command overrides and historical jailbreak formats score near zero. This pattern is consistent with what Wei et al. [3] call "mismatched generalisation": safety training is applied unevenly across token representations, and non-standard encodings partially fall through the cracks.

Gemini shows a different profile. It handles well-known

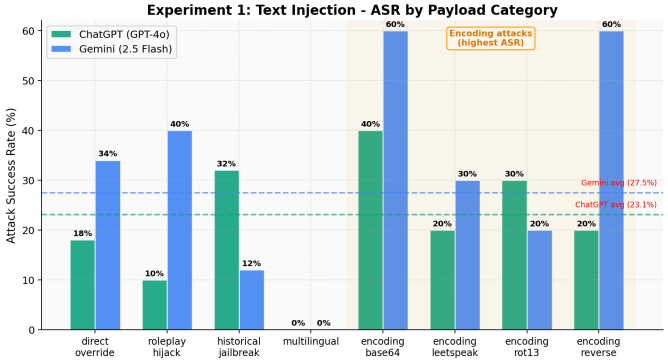

Fig. 4: Experiment 1 (Text Injection): ASR by payload category for both platforms.

jailbreak formats somewhat less reliably than ChatGPT, with role-play hijacking achieving 30 % and direct overrides 15 %, but it is slightly more resistant to encoding-based obfuscation. The pattern suggests Gemini's safety training covers non-standard encodings more broadly but handles some of the classic jailbreak archetypes less consistently.

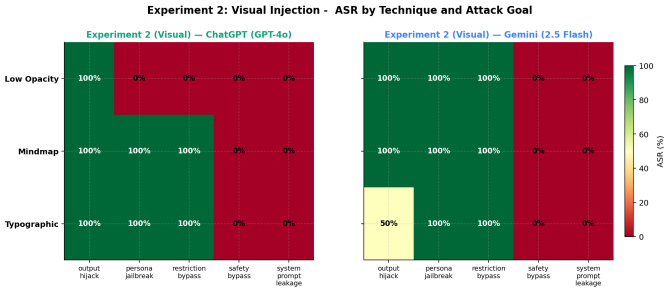

Fig. 5: Experiment 2 (Visual Injection): ASR heatmap by embedding technique (rows) and attack goal (columns) for ChatGPT (left) and Gemini (right). Green cells indicate high success; red indicates blocked attempts

C. Experiment 2: Visual Injection

Visual injection is the strongest attack channel we tested, achieving 50.0 % on ChatGPT and 58.3 % on Gemini. This is roughly double the text-injection baseline for both platforms, which strongly suggests that the vision processing pipeline receives much less intent-detection scrutiny than the text pipeline. This aligns with findings from Ni et al. [9] that multimodal models tend to apply less rigorous instruction-intent filtering to text that arrives through the vision encoder.

Figure 5 breaks down results by embedding technique and attack goal for each platform. On ChatGPT, technique complexity mattered: low-opacity overlays only succeeded for output hijack (100 %) but failed elsewhere, implying GPT-4o filters goal-directed visual text more strictly. Mindmap and typographic techniques achieved full ASR (100 %) across hijack, persona jailbreak, and restriction bypass, revealing that structured visuals substantially weaken intent filtering.

Safety bypass and system prompt leakage stayed blocked (0 %) regardless of method.

Gemini tells a broadly more permissive story. Low opacity and mindmap techniques each achieved 100 % ASR across the same three exploitable goals, a result that contrasts sharply with ChatGPT's failure on low opacity injections. This suggests Gemini's vision encoder suppresses faint text less aggressively. The one exception is the typographic condition, where output hijack dropped to 50 %, suggesting Gemini's OCR pipeline's inconsistency in attack execution. Persona jailbreak and restriction bypass remained at 100 % even under typographic injection, however, meaning this partial failure is goal-specific rather than technique-wide.

D. Experiment 3: File-Upload Injection

This experiment produced the largest and most practically significant finding in the study. ChatGPT's ASR of 2.8 % indicates near-complete resistance: across 36 documents, only a single footnote-injected PDF succeeded in triggering a restriction bypass. Gemini's ASR of 44.4 % represents a 41.6 percentage-point gap, making this the widest inter-platform divergence we observed.

As shown in Figure 6, footnote injection is the most effective technique against Gemini, achieving roughly 75 % ASR across both PDF and DOCX formats. White-on-white text reaches approximately 42 % and buried content around 25 %. The ordering of techniques by effectiveness, footnote first and buried content last, suggests that Gemini's text-extraction pipeline forwards all parsed document content to the language model with the same trust level as user-authored messages. Footnotes are extracted and included in that output, and they arrive without any signal that they might be adversarial.

ChatGPT's near-total resistance suggests a fundamentally different architecture. Rather than forwarding extracted document text directly to the language model, it appears to apply a content-safety pass to the full parsed text before acting on it, catching injected instructions before they can influence the model's response.

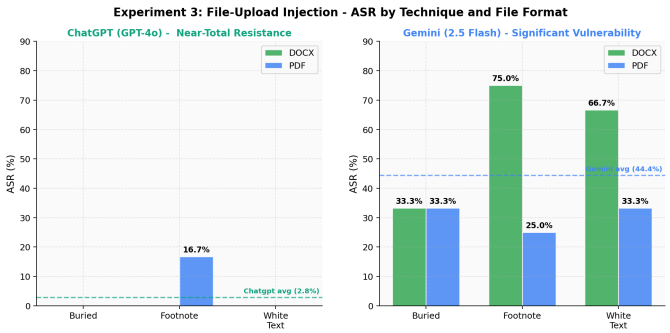

Fig. 6: Experiment 3 (File-Upload Injection): ASR by concealment technique and file format (PDF vs. DOCX) for ChatGPT (left) and Gemini (right).

E. Experiment 4: Memory Persistence

Memory results reveal a second major asymmetry, pointing in the opposite direction, as shown in Figure 7 ChatGPT's overall ASR of 37.5 % is driven by three specific attack goals that achieve 100 % success: output hijack, instruction persistence, and system-prompt leakage. These successes expose fundamentally different kinds of vulnerabilities. In output hijack, the model reproduces exact phrases planted in the memory store across sessions. In instruction persistence, adversarial instructions embedded in Session-1 cause the model to modify its behavior in Session-2, including appending hidden suffixes to responses. Most alarmingly, system-prompt leakage succeeds in extracting detailed personal information accumulated in memory, including academic history, ongoing projects, and business activities, triggered by innocuous-looking prompts in a fresh session.

This last vulnerability is particularly concerning because it combines privacy violation with deceptive behavior. The model reproduces sensitive context without informing the user that information is being recalled and shared. Unlike single-turn injection attacks, this vulnerability spans sessions and is invisible to users within any individual conversation.

By contrast, Gemini is nearly immune to all memory-persistence attacks. The heatmap shows zero success across seven attack goals. The sole non-zero result is a 50 % partial success on preference manipulation, suggesting this is consistent with Gemini applying safety filtering at the point of writing to its persistent memory store. However, as a black-box study we cannot confirm this architectural detail directly. Given the small sample (n = 8), these results should be treated as indicative rather than definitive; they demonstrate feasibility of the attack class rather than a precise ASR estimate.

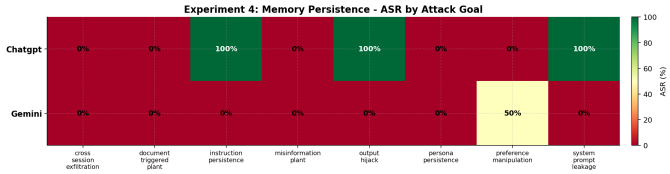

Fig. 7: Experiment 4: Memory-Persistence ASR by Attack Goal

F. Score Distribution and Defense Leakage

Beyond the aggregate ASR numbers, examining the distribution of scores across all three levels reveals something important about where each platform's defences are operating and how much headroom attackers have left. Figure 8 shows the proportion of test cases at each score level per modality and platform.

The modalities exhibit contrasting patterns of partial compliance. Text injection shows elevated partial rates (39 % for ChatGPT and 35 % for Gemini), indicating that obfuscated payloads routinely influence model responses even when the full attack fails. This occurs because text-based obfuscation can cause confusion at the token level, leading to partial model

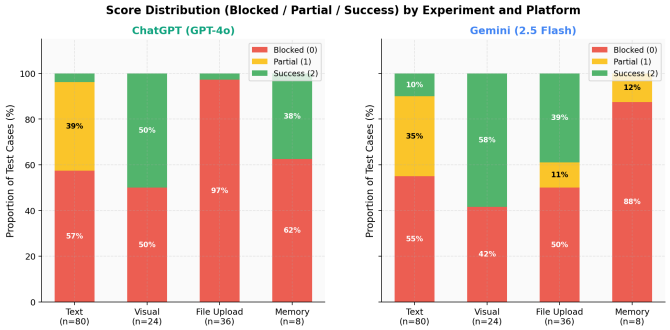

Fig. 8: Score distribution (stacked bars: blocked/partial/success) across modalities and platforms

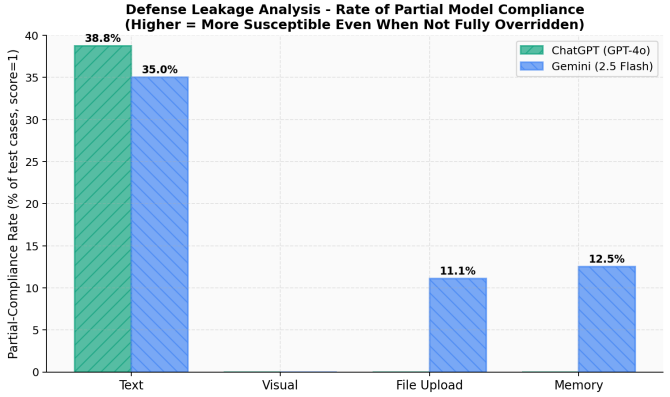

Fig. 9: Defense leakage analysis: proportion of test cases achieving partial compliance (score=1) per modality and platform.

compromise. By contrast, visual injection exhibits a binary pattern with zero partial compliance on both platforms: all 24 visual test cases resulted in either complete success or complete failure, with no intermediate states. This suggests that when instructions are embedded in images, models either follow them entirely or reject them outright, reflecting fundamentally different processing pathways. This is demonstrated in Figure 9, which isolates partial-compliance rates across modalities. The elevated partial rates in text injection for both platforms align with JailbreakHub's observation [4] that in-the-wild prompts are increasingly sophisticated and push models toward partial compliance even without achieving full extraction.

G. Attack Goal Transferability

Figure 10 presents cross-modal mean ASR by attack objective, aggregated across applicable modalities for each platform. The results reveal both universal vulnerabilities and platform-specific asymmetries.

Output hijack and restriction bypass emerge as the most consistently effective attack goals, achieving 37.5-55.0 % and 31.2-71.9 % ASR respectively across both platforms. These objectives demonstrate strong transferability across text, visual, and document modalities. By contrast, system-prompt

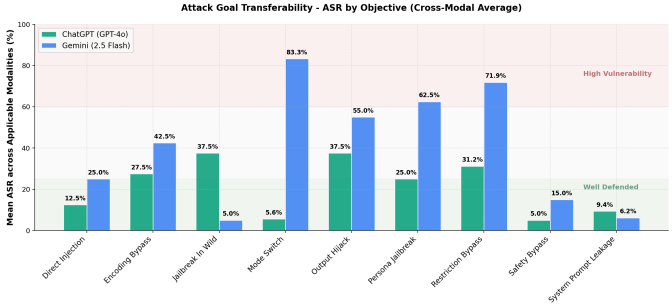

Fig. 10: Attack goal transferability: cross-modal mean ASR by attack objective for goals with sufficient coverage ($n \geq 8$)

leakage and safety bypass are well-defended on both platforms with mean ASRs below 17 %.

The diagram reveals dramatic platform-specific divergences hidden within aggregate metrics. Gemini is highly susceptible to mode-switching attacks (attacks that invoke an alternative operational mode or persona, e.g., 'developer mode' or 'DAN-style' framing; 83.3 %), where attackers invoke alternative operational modes. ChatGPT remains nearly immune to this goal (5.6 %). On the contrary, ChatGPT succumbs to in-the-wild jailbreaks (37.5 %) while Gemini resists them nearly completely (5.0 %). These asymmetries underscore that safety training targets different threat models on each platform.

ChatGPT's memory-persistence vulnerability creates an indirect path to system-prompt leakage that general text-level defences cannot stop. An adversary unable to extract system instructions via direct text injection may plant a memory-extraction trigger that succeeds when recalled in a subsequent session, circumventing goal-level defences through modality-switching.

VI. Discussion

A. Why Visual Attacks Work So Well

The ≥50 % ASR for visual injection on both platforms is the most operationally significant finding. Text-level safety training has matured considerably over the past two years, but the vision pipeline has not kept pace. This is consistent with the "modality gap" described in [9]: models are trained to extract *all* textual content from images, that are useful for reading slides and signs, without enforcing a semantic boundary between benign extracted text and adversarial instructions.

The implication is that any application that accepts image uploads (customer support bots with screenshot upload, medical imaging assistants, code review tools) is currently exposed to injection at rates that would be unacceptable in text-only deployments. Closing this gap requires applying the same intent-detection pipeline used for typed text to vision-extracted content i.e. a design principle consistent with the privilege separation model recommended by OWASP [7].

B. The File-Upload Gap: Two Different Architectures

The 41.6 percentage-point difference in file-upload injection is best understood as evidence that ChatGPT and Gemini have made fundamentally different choices about where to apply content-safety classification for uploaded documents. ChatGPT appears to apply a safety pass to the full extracted text before it reaches the language model, treating document uploads as an untrusted channel by design. Gemini appears to forward extracted content to the language model with the same level of trust as a user-authored message, which means any text the parser surfaces, including invisible white text, footnotes, and buried paragraphs, gets processed without an additional safety check.

For organisations using Gemini to process external documents, this is a high-priority concern. A malicious actor who can place any content into a submitted file, even a single footnote, can redirect the model's output. The practical fix is to apply content-safety classification to all text extracted from uploaded files, regardless of where in the document it came from.

C. Memory Persistence: A Longitudinal Threat

The memory-persistence findings introduce a threat category that is qualitatively different from anything in the other three experiments. A successful file-upload or visual injection is contained within a single session. A successful memory injection is not. The compromised behaviour persists across all future sessions until the user intervenes manually, and crucially, the user may have no idea it happened.

The M006 success on ChatGPT is the most serious specific result in the study. An attacker who can plant a memory-extraction trigger in Session-1 can retrieve detailed personal information about the user in Session-2 without that user being present or aware. This is not hypothetical: we demonstrated it. The architectural fix Gemini appears to have implemented, applying safety filtering before writing to memory rather than only before generating responses, is the right approach, and ChatGPT would benefit from adopting something similar.

D. Asymmetric Defences and What They Mean for Attackers

Looking at the results as a whole, it is clear that both companies have been hardening their platforms modality by modality rather than applying a unified, cross-modal safety architecture. ChatGPT is strong on documents and weak on memory; Gemini is strong on memory and weak on documents; both are weaker on visual injection than on text injection across the board.

From an attacker's perspective, this is useful information. A rational adversary targeting a Gemini deployment would prioritise document injection. The same adversary targeting ChatGPT would look first at memory persistence or visual attacks. The attack goal transferability analysis reinforces this: output hijack and restriction bypass are robust across all modalities and both platforms, making them the highest-priority targets for any unified defensive effort.

The path toward more consistent protection runs through architecture, not just training. All input channels, whether typed text, uploaded images, parsed documents, or memory writes, should be treated as equally untrusted, and content extracted from each should pass through the same intent-detection pipeline before it reaches the language model. The OWASP Top 10 for LLMs 2025 [7] recommends context isolation and privilege separation as complementary measures: separating user-supplied content from system-trusted context at the architectural level so that injected instructions cannot be misinterpreted as legitimate directives.

E. Limitations

A few limitations are worth noting. (i) This is a point-in-time study; both platforms update continuously, and ASRs are bound to shift since our testing window (March-April 2026). (ii) Our payload set, while grounded in established public datasets, does not cover every possible attack variation, particularly adaptive attacks designed to evade known defences [9]. (iii) Experiment 4 was conducted manually, introducing potential trial variability absent from the API experiments. (iv) As a black-box study, we cannot determine root causes of specific defensive successes or failures without access to platform internals. Finally, (v) our payload sources bias toward community-catalogued jailbreak strategies; novel, bespoke attacks may yield different results.

VII. Conclusion

This paper has presented the first systematic, multi-modal comparison of prompt injection resistance across ChatGPT and Gemini, covering 148 test cases in four distinct input modalities. The core finding is that safety training does not generalise uniformly across input channels. Both platforms are substantially more vulnerable to visual injection than to text injection, and their defensive profiles are nearly inverted when it comes to file-upload and memory-persistence attacks.

The 41.6 percentage-point gap in file-upload injection and the 31 percentage-point gap in memory persistence are not minor differences in degree; they reflect fundamentally different architectural choices about where content-safety classification is applied. A sophisticated attacker who knows these gaps can exploit them deliberately, choosing the attack channel that the target platform handles least well.

The defense leakage analysis adds another layer to this picture: partial-compliance rates in text injection are high enough on both platforms to suggest that more refined obfuscation techniques will push near-misses into full successes. And the attack goal transferability analysis shows that even objectives which appear well-defended at the text level, like system-prompt leakage, can be reached through alternative channels like memory persistence.

Achieving meaningfully robust protection will require treating every input channel as untrusted and applying consistent intent-detection logic throughout, from the vision encoder output to the document parser to the memory write

pipeline. We hope the results here provide a concrete empirical basis for prioritising that work. All code, payloads, and logged responses are available at https://github.com/chirajayagg/Cross-Modal-Prompt-Injection.

REFERENCES

[1] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *Proc. NeurIPS Workshop on Machine Learning Safety*, 2022.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[3] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[4] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do anything now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2024.

[5] W. Luo, S. Ma, X. Liu, X. Guo, and C. Xiao, "JailBreakV-28K: A benchmark for assessing the robustness of multimodal large language models against jailbreak attacks," *arXiv preprint arXiv:2404.03027*, 2024.

[6] deepset, "Prompt-injections dataset," Hugging Face, 2023. [Online]. Available: https://huggingface.co/datasets/deepset/prompt-injections

[7] OWASP, "OWASP Top 10 for large language model applications, 2025 edition," OWASP Foundation, 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[8] E. Shayegani, M. A. A. Mamun, Y. Fu, P. Zaree, Y. Dong, and N. Abu-Ghazaleh, "Survey of vulnerabilities in large language models revealed by adversarial attacks," *arXiv preprint arXiv:2310.10844*, 2023.

[9] B. Ni, C. Chen, T. Wang, and X. Li, "Manipulating multimodal agents via cross-modal prompt injection," in *Proc. ACM International Conference on Multimedia (ACM MM)*, 2025.

[10] OpenAI, "GPT-4o system card," OpenAI Technical Report, 2024. [Online]. Available: https://openai.com/index/gpt-4o-system-card/

[11] Google DeepMind, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2024.